

Lambda詳細設計書（実務版）

本設計書は、釣行AIアプリにおけるAWS Lambda関数単位の責務・入出力・処理フローを定義する。

1. 設計方針

- 1 Lambda = 1 責務（単一責任原則）
- API系とバッチ系を分離
- DynamoDBアクセスは原則Lambda経由
- リアルタイム処理は禁止（バッチ中心）

2. Lambda一覧（API系）

関数名	役割	トリガー	認証 【2026-07-26追加】
getRecommendationsHandler	TOP3取得	API Gateway	不要
getSpotsHandler	スポット一覧	API Gateway	不要
getPostsHandler	釣果取得	API Gateway	不要
createPostHandler 【2026-07-24 追加】	釣果投稿の作成	API Gateway（POST /posts）	必須
updatePostHandler 【2026-07-26 追加】	釣果投稿の編集 （本文・釣れた魚種・写真）	API Gateway（PUT /posts/{postId}）	必須

deletePostHandler 【2026-07-25 追加】	釣果投稿の削除	API Gateway (DELETE /posts/{postId})	必須
createFavoriteHandler	お気に入り登録	API Gateway	必須
getFavoritesHandler	お気に入り取得	API Gateway	必須
updateSpotImageHandler 【2026-07-24 追加】	スポット写真URLの設定	API Gateway (PUT /spots/{spotId}/image)	必須
presignUploadHandler 【2026-07-24 追加】	S3アップロード用の署名付きURL発行	API Gateway (POST /uploads/presign)	必須
postChatHandler 【2026-07-25 追加】	AIチャットへのメッセージ送信・Claude応答生成 (2026-07-26: 1日あたり利用回数の上限チェックを追加、GeminiからClaude Haikuへ移行)	API Gateway (POST /chat)	必須
getChatsHandler 【2026-07-25 追加】	チャット履歴一覧取得	API Gateway (GET /chats)	必須
getChatHandler 【2026-07-25 追加】	特定チャットの全メッセージ取得	API Gateway (GET /chats/{chatId})	必須
deleteChatHandler 【2026-07-25 追加】	チャットの削除	API Gateway (DELETE /chats/{chatId})	必須
getMyProfileHandler 【2026-07-26 追加】	自分の表示名（プロフィール）取得	API Gateway (GET /me)	必須
putMyProfileHandler 【2026-07-26 追加】	表示名の設定・更新	API Gateway (PUT /me)	必須

3. Lambda一覧（バッチ系）

関数名	役割	トリガー
generateSpotScoreBatch	天気・潮汐取得+AIスコア生成 （実装では1関数に統合）	EventBridge（毎日 AM6:00 JST）／ adminRunAiBatch経由
adminRunAiBatch	フロントの「AI分析を実行」ボタ ンからgenerateSpotScoreBatchを 起動する薄いラッパー。2026-07- 26: 1日3回のレート制限を追加 （コスト保護）	API Gateway（POST /admin/run-ai-batch、 2026-07-26～認証必須）
discoverSpotsBatch 【2026-07-24 追加、同日 中に役割変更】	Google Places APIで新規釣りスポ ット候補を探索しSpotsテーブルへ 追加。2026-07-26: 釣具店 （Placesのtypesに"store"を含む候 補）を除外するようになった	adminRunSpotDiscovery 経由のみ（自動スケジ ュールは廃止）
adminRunSpotDiscovery 【2026-07-24 追加】	フロントの「現在地から新スポッ トを探す」ボタンから discoverSpotsBatchを起動する薄 いラッパー（adminRunAiBatchと 同一コードを BATCH_FUNCTION_NAME環境変 数の差し替えで再利用）。2026-07- 26: 1日10回のレート制限を追加	API Gateway（POST /admin/run-spot- discovery、2026-07-26～ 認証必須）
searchTackleShops 【2026-07-26 追加】	釣具店をGoogle Places APIで検索 して返す（DB書き込みなし・スコ アなし）	API Gateway（POST /tackle-shops/search、認 証必須）

【2026-07-23 実装更新】設計当初は fetchWeatherBatch / fetchTideBatch /
fetchFishingReportsBatch / updateRecommendationBatch の4分割を想定していたが、実装
では generateSpotScoreBatch 1関数に統合している（スポットごとに天気・潮汐取得→ス
コア計算→AIコメント生成→DynamoDB保存までを直列実行）。釣果収集
(fetchFishingReportsBatch＝外部サイトからの釣果自動収集)は信頼できる無料データソー
スが無い場合 引き続きスコープ外・未実装。**ユーザー自身による釣果投稿機能**

(createPostHandler) は2026-07-24に実装済みで、これは fetchFishingReportsBatch とは別物（外部収集ではなくユーザー投稿）。

【2026-07-24 実装更新・午前】これまで generateSpotScoreBatch はSpotsテーブルの既存スポットをre-scoreするのみで、新規スポットを増やす手段が無かった（seed_data.pyの手動編集・再実行のみ）。discoverSpotsBatch を追加し、Google Places API のText Search（実在POIデータ）で新規候補を探索、既存スポットと300m以内なら重複除外した上でSpotsテーブルへ自動追加するようにした。想定される魚種はClaude APIで地名から推測するが、座標・名称などの実在性に関わる情報はPlaces APIの結果のみを採用し、LLMには生成させない（座標のハルシネーションを避けるため）。

【2026-07-24 実装更新・同日中に再変更】ユーザーからのフィードバックにより、「新スポットを探す」を独立ボタンのまま 残すのではなく、**generateSpotScoreBatch が discover_spots.run_discovery() を内部で呼び出す形に統合**した。「AI分析を実行」ボタンを押すたびに (a) 位置指定なしの全国向け探索 → (b) 全スポットのスコア計算、の順に実行される。これに伴い discoverSpotsBatch の週次EventBridgeスケジュールは冗長になったため廃止し、この関数は「現在地から新スポットを探す」ボタン（ユーザーの現在地に絞った近傍探索専用、半径15km、スコア計算はしない）専用として存続させている。

実装上は generate_score.py と discover_spots.py の相互import（循環import）を避けるため、共有ヘルパー（DynamoDB/SSMアクセス等）を新設の batch_common.py に切り出し、両者がそこから import する構成にした。

4. 個別設計

getRecommendationsHandler

目的：ユーザーにTOP3スポットを返却

入力

```
{
  "userId": "xxx",
  "lat": 35.6,
```

```
"lng": 139.7
}
```

処理フロー

1. Recommendations取得
2. 距離補正
3. スコア順ソート
4. TOP3抽出
5. レスポンス返却

出力

```
{
  "top3": [
    {
      "spotId": "A123",
      "score": 87,
      "fishTypes": ["アジ"],
      "reason": "潮流が安定",
      "distance": 3.2,
      "cost": 500
    }
  ]
}
```

generateSpotScoreBatch

目的：AIによる釣行スコア生成

処理

Spots取得
+ Weatherデータ (Open-Meteo Forecast API)
+ Tideデータ (Open-Meteo Marine API・海面水位)
→ スコア計算
→ Claude APIで推薦理由(reason)を生成
→ DynamoDB更新

天気・潮汐データソース【2026-07-23 実装】

外部の天気・潮汐APIは **Open-Meteo** (<https://open-meteo.com/>) を採用。APIキー登録不要・無料（非商用）で、複数の気象・海洋機関のデータを統合しており信頼性が高い。

- ・天気スコア：weather_code（WMO天気コード）・降水量・風速から独自ヒューリスティックで算出
- ・潮汐スコア：sea_level_height_msl（海面水位）の前後1時間の変化率から算出（潮が動いている時間帯ほど魚の活性が上がるという経験則に基づく）
- ・外部API呼び出しが失敗した場合はシミュレーション値にフォールバックし、バッチ全体は継続する

スコアロジック

```
score =  
    fish_probability * 0.4  
+ weather_score * 0.2  
+ tide_score * 0.2  
- distance_penalty * 0.1  
- cost_penalty * 0.1
```

【2026-07-26追加・reason使い回しによるAPI呼び出し削減】discover_spots.pyの新スポット探索や 釣具店除外により、SpotsTableのスポット数が当初の5件から100件超に増えていた。generate_score.py は元々スポットごとに**必ず**Gemini APIを1回呼んで推薦理由(reason)を生成していたため、「AI分析を実行」を1回押すだけでスポット数と同じ回数（100回超）Gemini APIを呼ぶことになり、Gemini無料枠の1日あたり上限（20リクエスト/プロジェクト）を1回の実行で使い切ってしまう 実害が発生した（AI相談が「回答の生成に失敗しました」を返す形で顕在化）。

対策として、_should_regenerate_reason(existing, new_score)を追加し、以下のいずれにも 該当しないスポットは前回のreason文をそのまま使い回し、AI呼び出しをしないようにした:

- ・ そのスポットの推薦データが存在しない（初回）
- ・ スコアが前回からREASON_SCORE_CHANGE_THRESHOLD（10.0）以上動いた
- ・ 前回の生成からREASON_MAX_AGE_DAYS（3日）以上経過している

スコア自体（天気・潮汐・DynamoDBのupdatedAt）は使い回し対象外で毎回必ず再計算・更新するため、フロント（useRecommendations.ts）の「全スポットのupdatedAtが実行開始時刻より新しくなったら完了」というポーリング判定には影響しない。結果として、2回目以降の「AI分析を実行」はスコアが大きく動いたスポットの分だけAI呼び出しをする形

になり、通常は数回～十数回程度で収まる想定。

【2026-07-26追加・Gemini→Claude Haiku移行】上記の対症療法だけでは、スポット数がさらに増えれば 再び無料枠を使い切るリスクが残る構造的な問題だった。そのため、AIコメント生成・AI相談チャット・魚種推測の3箇所すべてをGemini APIからClaude Haiku (Anthropic API) に移行した。Claudeには Geminiのような「1日20回」といった硬い無料枠の壁が無く、低単価の従量課金のみのため、回数上限を心配する必要がなくなった。SSMパラメータは/fishing-ai/anthropic-api-keyに、モデルIDはclaude-haiku-4-5-20251001にそれぞれ変更している。上記のreason使い回しロジック自体は、無駄なAPI呼び出し・コスト削減のためそのまま維持している。

adminRunAiBatch 【2026-07-23 追記】

目的：フロントの「AI分析を実行」ボタンから generateSpotScoreBatch を起動する

API Gateway (REST API) は統合タイムアウトが29秒固定であり、Claude API呼び出しを含む generateSpotScoreBatch (5スポット分) はこれを超えることがある。当初は同期呼び出し (RequestResponse) で実装していたため、Lambda自体は成功しているのに API Gateway が 504 timeoutを返し、ボタンにエラー表示が出る問題があった。

対応として adminRunAiBatch は generateSpotScoreBatch を**非同期呼び出し (Event)** に変更し、起動受付 (202) を即座に返す方式に変更した。完了確認はフロントエンド側で GET /recommendations を定期的にポーリングして行う。

discover_spots.run_discovery() 【2026-07-24 追加】

目的：新規釣りスポット候補をGoogle Places APIで探索し、Spotsテーブルに追加する中核ロジック。generateSpotScoreBatch のhandler冒頭 (位置指定なし) と、discoverSpotsBatch のhandler (「現在地から新スポットを探す」ボタン経由、位置指定あり) の両方から呼ばれる共通関数。

処理

Spots全件取得 (重複判定用)
→ Google Places API Text Search

- ・位置指定なし: "釣り 堤防" 等の全国向け複数クエリ
- ・位置指定あり: "釣り" 1クエリ + location/radius(15km)パラメータで近傍に絞り込み
- Placesのtypesに"store"を含む候補(釣具店等の小売店)を除外【2026-07-26追加】
- 既存スポットと緯度経度がhaversine距離300m以内なら重複として除外
- Claude APIで地名・住所から想定される魚種を推測(失敗時は汎用デフォルト)
- 基準地点(位置指定時はその座標、なければ東京駅)からのdistanceKmを算出
- Spotsテーブルへput_item

【2026-07-26追加】"釣り"というクエリだけでは上州屋等の釣具店(タックルショップ)が検索結果に混ざり込み、釣り場ではないスポットがおすすめに出現してしまう問題があった。Google Places APIが返す各結果の types 配列を保持しておき、"store"を含む候補はrun_discoveryのループ内でスキップするようにした(釣り場自体は通常point_of_interest/park/natural_feature等で"store"を持たない)。釣具店を探したいニーズ自体には、別途 tackle_shops.py (searchTackleShops) で応える。

座標・スポット名などの実在性に関わる情報はGoogle Places APIの結果のみを採用し、LLM (Claude) には生成させない(存在しない場所をAIが作り出すハルシネーションを避けるため)。Claudeは魚種の推測という、実在性に影響しない付随情報の生成にのみ用いる。

generate_score.py が discover_spots.py の run_discovery() を直接importして呼び出すため、逆方向のimport (discover_spots→generate_score) は循環importになり不可能。共有ヘルパー (DynamoDB/SSMアクセス・HTTP GET) は batch_common.py に切り出し、両モジュールがそこから import する。

presignUploadHandler / updateSpotImageHandler / createPostHandler

【2026-07-24 追加】

目的: スポット写真・投稿写真のアップロードと、釣果投稿の作成

処理フロー (写真アップロード) 【2026-07-25 方式変更】

1. フロントが POST /uploads/presign を呼び、S3への署名付きPOSTフォーム (uploadUrl + uploadFields、有効期限)
2. フロントがそのフォームへ画像ファイルをmultipart/form-dataでPOST (Lambda/API Gatewayを経由しない。ペー
3. アップロード完了後、公開URL (publicUrl) を
PUT /spots/{spotId}/image (スポット写真) または

POST /posts のimageUrl (投稿写真) または
POST /chat のimageUrl (チャット添付) として保存

アップロード先S3バケット (UploadsBucket) はGetObjectのみ公開許可、書き込みは署名付きURL経由のみに限定している。

【2026-07-25】当初は ``generate_presigned_url("put_object")`` による単純な署名付きPUT URLだったが、URL自体にサイズ制限を埋め込めずクライアントが任意サイズをアップロードできる欠陥があった。``generate_presigned_post`` に変更し、Conditions に ``["content-length-range", 1, 8388608]`` (8MB) を指定することで S3側にアップロード上限を強制させている。

postChatHandler / getChatsHandler / getChatHandler 【2026-07-25 追加】

目的：写真添付にも対応したAIチャット機能。「カメラ機能」と「チャット機能」を別々に作る案もあったが、ユーザーの意向により写真添付（端末アルバムから選択 or その場で撮影、どちらも同じ `<input type="file">` で実現し、撮影用は `capture="environment"` 属性を付与）をチャット入力の一部として統合した。

処理フロー (POST /chat)

1. chatId が未指定なら新規チャットとしてuuid発行、指定されていればChatsTableから既存messagesを読み込み
2. ユーザーメッセージ (+ imageUrlがあればS3から画像バイトを取得) をmessagesに追記
3. Claudeへ直近10往復分の会話履歴 + 今回のメッセージ (+画像) をマルチモーダル入力として送信
(システムプロンプトで「釣りの専門家アシスタント」という役割を明示)
4. 応答をmessagesに追記し、ChatsTableへput_item (アイテム全体を上書き)
5. { chatId, reply, updatedAt } を同期的に返却

他のAI機能 (generateSpotScoreBatch等) は API Gateway の29秒タイムアウトを避けるため 非同期トリガー+ポーリング方式を採っているが、チャットはUX上ポーリングが合わないため **同期呼び出しのまま** とし、Lambda Timeoutを29秒より短い25秒に設定することで対処した (テキスト+画像1枚のClaude呼び出しは通常数秒～十数秒で収まる想定)。

GET /chats は履歴パネルの一覧表示用に messages を含まない軽量のレスポンス (chatId/title/updatedAtのみ) を返す。GET /chats/{chatId} は履歴から会話を再開する際に

全メッセージを返す。「新しい会話」開始時はDBに何も書き込まず、次にメッセージを送信した時点で初めて新規チャットとして保存される。

【2026-07-26追加】友人にアプリを共有し複数人が使うようになったことで、AI API呼び出し（＝課金）が想定外に増えるリスクが生じた。認証実装により実ユーザーIDが取得できるようになったことを利用し、postChatHandlerの冒頭でClaude呼び出し前に `_check_and_increment_daily_usage(user_id, "chat", DAILY_CHAT_LIMIT)` を呼ぶようにした。UsageTable（userId + dateKey複合キー、dateKeyは"chat#2026-07-26"の形式）へのupdate_item（ADD式によるアトミック加算）で当日のカウントを取得・加算し、1日30件（DAILY_CHAT_LIMIT）を超えていれば429（{"error": "rate_limited", ...}）を返してClaudeを呼ばずに終了する。UsageTableの各アイテムにはTTL（翌々日ごろ）を設定しており、DynamoDB側で自動的に古いカウンタが削除されるため、明示的なクリーンアップ処理は不要。

【2026-07-26追加・実データグラウンディング】「近くの釣り場は？」「〇〇県のおすすめは？」といった質問に、AIが実際に登録されているデータで答えられるようにした。discover_spots.pyが「座標などの実在情報はPlaces APIのみを採用しClaudeに生成させない」という方針を取っているのと同じ考え方で、Function calling等の複雑な仕組みは使わず、_build_spots_context(lat, lng) がSpots・Recommendationsをscan→結合→（現在地が分かれば）距離順、（分からなければ）スコア降順で上位40件（SPOTS_CONTEXT_LIMIT、トークン量＝コストの際限ない増加を防ぐ上限）をテキスト化し、system_instructionに埋め込んで毎回渡す方式にした。「データに無い場所は正直に 無いと伝え、存在しない場所を作り出さない」という指示も明記しハルシネーションを防いでいる。フロント側（ChatInput.tsx）の現在地共有トグルをONにすると、POST /chatのbodyにlat/lngが乗り、距離を実際に計算（haversine）した上で近い順の案内ができる（トグルOFFでもプレフェクチャ名等のテキストだけでコンテキストは機能する）。

updatePostHandler 【2026-07-26追加】

目的：釣果投稿の編集（本文・釣れた魚種・写真）。所有者チェックはdeletePostHandlerと全く同じ方式（PostsTableはpostId単一PKで所有者情報がキーに無いため、更新前にget_itemして投稿のuserIdとリクエスト元のuserIdを比較し、一致しなければ403を返す）。

リクエストボディに含まれるフィールドのみを更新する（部分更新）。contentが空文字の場合は400を返す。投稿先スポット（spotId）はAPI側では変更を受け付けない（フロントの

PostForm.tsxも編集モードではスポット選択欄を無効化している)。更新のたびにupdatedAtを記録し、フロント側で「(編集済み)」の表示に使う。

コスト保護レート制限 (admin_trigger.py / tackle_shops.py) 【2026-07-26追加】

目的：AI分析実行・現在地から新スポット探索・釣具店検索はいずれもPlaces/Claude APIの呼び出しを伴い課金対象のため、AI相談 (postChatHandlerのDAILY_CHAT_LIMIT) と同じ思想で1ユーザー1日あたりの上限を設けた。api/とbatch/はSAMの別デプロイパッケージのため handlers.pyのロジックは共有できず、batch_common.pyに get_user_id・check_and_increment_daily_usageとして複製した (discover_spots.pyのhaversine_km等、既存の重複パターンを踏襲)。

関数	レート制限アクション名	1日の上限	設定方法
adminRunAiBatch	ai-batch	3回	template.yamlのRATE_LIMIT_ACTION/RATE_LIMIT_DAILY環境変数
adminRunSpotDiscovery	spot-discovery	10回	同上 (同じadmin_trigger.pyを異なる環境変数で再利用)
searchTackleShops	tackle-shop-search	20回	tackle_shops.pyのDAILY_SEARCH_LIMIT定数

上限に達している場合はバッチ起動・Places API検索を一切行わずに429 ({ "status"/"error": "rate_limited", "message": "..."}) を返す。カウンタはUsageTable (userId + dateKey) にADD式でアトミック加算し、TTLで翌々日ごろ自動的に古いカウンタが削除される (postChatHandlerと全く同じ仕組み)。

getMyProfileHandler / putMyProfileHandler 【2026-07-26追加】

目的：ユーザーが自分の表示名を設定・編集できるようにする。ナビにメールアドレスが出るのが微妙という声と、釣果投稿に投稿者名を出したいという要望を受けて追加した。

UsersTable (userId単一PK) に displayName・email・createdAt・updatedAt を保存する。GET /me は未登録でも404にせず displayName: null で200を返す（フロント側で「未設定」の判定に使う一次情報として扱いやすくするため）。

投稿一覧 (getPostsHandler) は、SpotsTable/FavoritesTableの結合と同じ「NoSQLはJOINできないので アプリ側でuserId→表示名の辞書を作って結合する」パターンでUsersTableをscanし、各投稿に authorName（未設定ユーザーは"匿名"）を付与する。投稿時点でフリーズさせず、表示のたびに最新の表示名を反映する設計（表示名を後から変更すると過去の投稿の表示も更新される）。

tackle_shops.py (searchTackleShops) 【2026-07-26追加】

目的：釣具店（タックルショップ）専用の検索機能。従来はdiscover_spots.pyの"釣り"クエリに 釣具店が混入していたため、EXCLUDED_PLACE_TYPES ("store") で除外する一方、釣具店を探したい ニーズ自体には応える必要があった。

処理

```
POST /tackle-shops/search { lat?, lng?, keyword? }
→ discover_spots.py の search_places("釣具店 " + keyword, api_key, location_bias) を再利用
→ lat/lng指定時: 各結果との距離(haversine_km)を計算し近い順にソート
→ 最大MAX_RESULTS(20)件を { name, lat, lng, address, distanceKm? } の形で返す
→ DynamoDBへの書き込みは一切行わない（都度検索するだけ、スコアも付与しない）
```

discoverSpotsBatchと同じ ../backend/lambda/batch/ パッケージに配置し、discover_spots.py の search_places/haversine_km をそのままimportして再利用している（コードの重複を避けるため）。Places API呼び出しは課金を 伴うため、POST /tackle-shops/search はCognito認証必須にしている。

5. 非機能要件

- API応答：1～2秒以内
- バッチ：1時間単位実行

- エラーハンドリング：リトライ設計

6. セキュリティ

- IAM最小権限
- Cognito認証必須（操作系エンドポイントのみ）
- API Gateway認証制御

【2026-07-24 時点・経緯】Cognito認証は当初未実装で、個人利用中心を想定し `userId` は "user-001" 固定で運用していた。他の機能（現在地からのおすすめ・画像機能・投稿・新規スポット自動発見）を先に実装したうえで最後に対応する方針としていた。

【2026-07-26 実装】友人にアプリを共有した結果、固定 `userId` の限界（全員のお気に入り・AI相談履歴が混ざる、投稿削除に所有者チェックが無い）が実害になったため、Cognito User Pool + Google IdP（Hosted UI）による Google 認証を実装した。要件定義.htmlに明記の「セキュリティ：Google 認証（Cognito）」に対応する。

6.1 認証範囲

閲覧は未ログインでも可能（おすすめ・地図・スポット一覧・投稿一覧）。**操作（お気に入り・投稿作成/削除・AI相談・AI分析実行・新スポット探索）はログイン必須**という方針とした（誰でも見られるアプリという性質を保ちつつ、書き込み系だけを保護する狙い）。

6.2 認証フロー

フロント (react-oidc-context, Authorization Code + PKCE)

- Cognito Hosted UI (/oauth2/authorize) へリダイレクト
- Hosted UI上でGoogleログイン (Cognito側のGoogle IdP連携経由)
- Cognitoの/oauth2/idpresponseへコールバック → 認可コードを `id_token` に交換
- 以後のAPIリクエストに `Authorization: Bearer <id_token>` を付与
- API Gateway の CognitoAuthorizer (User Pool Authorizer) が `id_token` を検証
- 検証成功時、Lambda側イベントに `event.requestContext.authorizer.claims.sub` が入る
- `handlers.py` の `_get_user_id(event)` がこれを実ユーザーIDとして使用

CognitoはOIDC標準の `end_session_endpoint` をディスカバリドキュメントに含めないため、ログアウトは `react-oidc-context` の標準 `signoutRedirect()` ではなく、Hosted UI独自の `/logout`

エンドポイントへ直接遷移する方式で実装している（frontend/src/auth/authConfig.ts の `cognitoSignOut()`）。

【2026-07-26・revert済み】Hosted UIのドメイン（`UserPoolDomain.Domain`）を、見栄えの良い短いプレフィックス（`ryu-chan-fish`、Cloudflare Workers側の サブドメインとブランドを揃えたもの）に変更しようとしたが、デプロイが失敗した。

原因：CloudFormationはDomainプロパティの変更を「新しいリソースを作成してから古いものを削除する」置き換えとして実行しようとするが、Cognitoの実際のAPIは1つのUserPoolに同時に2つのドメインを持てないため、新ドメインのCreateUserPoolDomainがInvalid request providedで拒否される。CloudFormation側は安全にロールバックし、スタックは元の状態（アカウントID入りドメイン: `fishing-ai-app-<AWSアカウントID>.auth...`）に復旧済み。

短いドメインへの変更には「①ドメインを一旦削除するデプロイ→②新ドメインを追加するデプロイ」の2段階が必要で、①～②の間はログインが機能しなくなる（Google Cloud Console側のリダイレクトURI 変更ともタイミングを合わせる必要がある）ため、ユーザーと調整の上で改めて実施する（現状はアカウントID入りドメインのまま据え置き）。

6.3 所有者チェック

Favorites/Chatsテーブルは `userId` がpartition keyの一部のため、他人の`userId`ではそもそも該当データにアクセスできない設計（追加の所有者チェック不要）。一方Postsテーブルは `postId` 単一PKで所有者情報がキーに含まれないため、`deletePostHandler`で削除前に `get_item` して投稿の `userId` と リクエスト元の `userId` を比較し、一致しなければ403を返す明示的なチェックを追加した。

7. テスト戦略【2026-07-25 追加】

- バックエンド: pytest。純粋関数（`calc_score`・`haversine_km`等）の単体テストに加え、`moto` でDynamoDBをモックしたハンドラーレベルのテスト（投稿・チャットの作成→削除ライフサイクル等）
- フロントエンド ユニットテスト: Vitest + React Testing Library（utils/の純粋関数、RecommendationCardのレンダリングテスト）

- フロントエンド E2E・VRT: Playwright。実AWSには接続せず `page.route()` でAPIレスポンスをモックする。VRT (`toHaveScreenshot`) のベースライン画像はCI環境で生成する必要がある (OS依存のため。詳細はREADME参照)
- CI: `.github/workflows/test.yml` が `pull_request → main` をトリガーに実行される (`deploy.yml` の `push → main` トリガーとは分離。デプロイは行わずテストのみ)

AIによる自動コードレビュー (Claude API等) はPRごとの課金が発生するため今回は見送った。GitHub標準のPRレビュー機能 (コメント・Approve/Request changes) は追加設定なしで利用できる。

8. まとめ

Lambdaは「API処理」と「バッチ処理」に完全分離し、スケーラブルかつ低コストな構成とする。